

Unidad III: Funciones map y manejo de errores en iteraciones

Denis Javier Rivera Arbaiza

Contents

Introducción	2
1. Las funciones map	2
1.1 La familia de funciones <code>map</code>	3
1.2 <code>map()</code> devuelve una lista	4
1.3 <code>map_dbl()</code> devuelve un vector numérico	5
1.4 <code>map_chr()</code> devuelve texto	6
1.5 <code>map_lgl()</code> devuelve valores lógicos	6
1.6 <code>map_int()</code> devuelve enteros	7
1.7 Uso de funciones anónimas	7
1.8 Pasar argumentos adicionales	8
1.9 Extraer elementos por nombre	9
1.10 Comparación entre <code>for</code> , <code>lapply()</code> y <code>map()</code>	12
2. Manejo de errores al iterar	13
2.1 La función <code>safely()</code>	14
2.2 Usar <code>safely()</code> con <code>map()</code>	15
2.3 Separar resultados y errores	16
2.4 Identificar qué elementos funcionaron	17
2.5 La función <code>possibly()</code>	18
2.6 Diferencia entre <code>safely()</code> y <code>possibly()</code>	19
2.7 La función <code>quietly()</code>	19
2.8 Usar <code>quietly()</code> con <code>map()</code>	20
3. Ejercicios	22
Ejercicio 1	22
Ejercicio 2	22
Ejercicio 3	22
Ejercicio 4	23

Ejercicio 5	23
Ejercicio 6	23
Ejercicio 7	23
Ejercicio 8	24
Ejercicio 9	24
Ejercicio 10	25

```
library(purrr)
library(dplyr)
library(tibble)
```

Introducción

En los temas anteriores estudiamos los bucles **for** y las funciones de orden superior. Vimos que muchas tareas de programación consisten en repetir una misma operación sobre muchos elementos.

Por ejemplo:

- calcular la media de muchas columnas;
- obtener la clase de cada variable;
- contar valores únicos en cada columna;
- aplicar una función a cada elemento de una lista;
- ajustar varios modelos;
- generar muchas simulaciones.

En este documento estudiaremos dos temas importantes:

1. Las funciones **map**.
2. Manejo de errores al iterar.

Las funciones **map()** pertenecen al paquete **purrr** y permiten escribir iteraciones de una forma más compacta y clara. Además, veremos cómo evitar que un error en una sola iteración detenga todo el proceso.

1. Las funciones **map**

La idea principal de **map()** es sencilla:

aplicar una función a cada elemento de un objeto.

Por ejemplo, si tenemos varias columnas en una base de datos, podemos aplicar la misma función a cada columna.

Consideremos la base de datos **mtcars**.

```
head(mtcars)
```

```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0  1   4   4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0  1   4   4
## Datsun 710     22.8   4  108  93 3.85 2.320 18.61 1  1   4   1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1  0   3   1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0  0   3   2
## Valiant        18.1   6  225 105 2.76 3.460 20.22 1  0   3   1
```

Supongamos que queremos calcular la media de cada columna. Con un bucle `for`, podríamos escribir:

```
salida <- vector("double", length(mtcars))
names(salida) <- names(mtcars)

for (i in seq_along(mtcars)) {
  salida[[i]] <- mean(mtcars[[i]])
}

salida
```

```
##           mpg           cyl           disp           hp           drat           wt           qsec
## 20.090625    6.187500    230.721875    146.687500    3.596563    3.217250    17.848750
##           vs           am           gear           carb
##  0.437500    0.406250    3.687500    2.812500
```

La misma tarea puede escribirse con `map_dbl()`:

```
map_dbl(mtcars, mean)
```

```
##           mpg           cyl           disp           hp           drat           wt           qsec
## 20.090625    6.187500    230.721875    146.687500    3.596563    3.217250    17.848750
##           vs           am           gear           carb
##  0.437500    0.406250    3.687500    2.812500
```

La instrucción anterior significa:

toma cada columna de `mtcars`, aplícale la función `mean()` y devuelve un vector numérico.

1.1 La familia de funciones `map`

El paquete `purrr` tiene varias versiones de `map()`. La diferencia principal entre ellas es el tipo de salida que devuelven.

Función	Tipo de salida
<code>map()</code>	Lista
<code>map_lgl()</code>	Vector lógico
<code>map_int()</code>	Vector entero
<code>map_dbl()</code>	Vector numérico tipo double
<code>map_chr()</code>	Vector de texto

La estructura general es:

```
map(objeto, funcion)
```

o, dependiendo del tipo de salida:

```
map_dbl(objeto, funcion)
map_chr(objeto, funcion)
map_lgl(objeto, funcion)
map_int(objeto, funcion)
```

1.2 map() devuelve una lista

La función `map()` siempre devuelve una lista.

```
map(mtcars, mean)
```

```
## $mpg
## [1] 20.09062
##
## $cyl
## [1] 6.1875
##
## $disp
## [1] 230.7219
##
## $hp
## [1] 146.6875
##
## $drat
## [1] 3.596563
##
## $wt
## [1] 3.21725
##
## $qsec
## [1] 17.84875
##
## $vs
## [1] 0.4375
##
## $am
## [1] 0.40625
##
## $gear
## [1] 3.6875
##
## $carb
## [1] 2.8125
```

Aunque cada resultado sea un solo número, la salida se guarda como lista.

Podemos verificarlo:

```
resultado <- map(mtcars, mean)

class(resultado)
```

```
## [1] "list"
```

Esto puede ser útil cuando la salida de cada iteración no tiene necesariamente la misma longitud o el mismo tipo.

Por ejemplo:

```
x <- list(
  a = 1:5,
  b = c(10, 20),
  c = c(3, 4, 5, 6)
)

map(x, summary)
```

```
## $a
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##         1         2         3         3         4         5
##
## $b
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.0    12.5    15.0    15.0    17.5    20.0
##
## $c
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     3.00    3.75    4.50    4.50    5.25    6.00
```

Cada `summary()` produce un objeto con varios valores. Por eso es natural guardar todo en una lista.

1.3 `map_dbl()` devuelve un vector numérico

Cuando sabemos que cada resultado será un número decimal, usamos `map_dbl()`.

Por ejemplo, para calcular medias:

```
map_dbl(mtcars, mean)
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Para calcular medianas:

```
map_dbl(mtcars, median)
```

```
##      mpg      cyl    disp      hp    drat      wt      qsec      vs      am      gear
## 19.200   6.000 196.300 123.000   3.695   3.325  17.710   0.000   0.000   4.000
##      carb
##      2.000
```

Para calcular desviaciones estándar:

```
map_dbl(mtcars, sd)
```

```
##      mpg      cyl      disp      hp      drat      wt
## 6.0269481 1.7859216 123.9386938 68.5628685 0.5346787 0.9784574
##      qsec      vs      am      gear      carb
## 1.7869432 0.5040161 0.4989909 0.7378041 1.6152000
```

La ventaja de `map_dbl()` es que exige que cada resultado sea un número. Si alguna función devuelve algo que no puede convertirse en número, R mostrará un error.

Esto es bueno porque ayuda a detectar problemas.

1.4 map_chr() devuelve texto

La función `map_chr()` se usa cuando esperamos que cada resultado sea texto.

Por ejemplo, podemos obtener la clase de cada columna de `iris`.

```
map_chr(iris, class)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## "numeric" "numeric" "numeric" "numeric" "factor"
```

Aquí se aplica `class()` a cada columna de `iris`.

Como la salida de `class()` es texto, usamos `map_chr()`.

1.5 map_lgl() devuelve valores lógicos

La función `map_lgl()` devuelve un vector lógico, es decir, un vector formado por valores `TRUE` o `FALSE`.

Por ejemplo, podemos preguntar qué columnas de `iris` son numéricas.

```
map_lgl(iris, is.numeric)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## TRUE TRUE TRUE TRUE FALSE
```

El resultado indica, para cada columna, si es numérica o no.

También podemos preguntar qué columnas son factores.

```
map_lgl(iris, is.factor)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## FALSE FALSE FALSE FALSE TRUE
```

Esta instrucción es útil cuando queremos filtrar columnas.

Por ejemplo, podemos seleccionar solo las columnas numéricas de `iris`:

```
iris_num <- iris[map_lgl(iris, is.numeric)]
head(iris_num)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1 5.1 3.5 1.4 0.2
## 2 4.9 3.0 1.4 0.2
## 3 4.7 3.2 1.3 0.2
## 4 4.6 3.1 1.5 0.2
## 5 5.0 3.6 1.4 0.2
## 6 5.4 3.9 1.7 0.4
```

1.6 map_int() devuelve enteros

La función `map_int()` se usa cuando esperamos resultados enteros.

Por ejemplo, podemos contar cuántos valores diferentes tiene cada columna de `iris`.

```
map_int(iris, ~ length(unique(.)))
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 35 23 43 22 3
```

Aquí aparece una nueva forma de escribir funciones: la fórmula con `~`.

La expresión:

```
~ length(unique(.))
```

significa:

toma cada elemento, represéntalo con `.`, calcula sus valores únicos y cuenta cuántos hay.

En este caso, `.` representa la columna actual.

1.7 Uso de funciones anónimas

A veces no queremos aplicar directamente una función existente, sino una operación más personalizada.

Por ejemplo, queremos calcular el rango de cada columna de `mtcars`, es decir, máximo menos mínimo.

Con una función anónima tradicional:

```
map_dbl(mtcars, function(x) {
  max(x) - min(x)
})
```

```
##      mpg      cyl    disp      hp    drat      wt     qsec      vs      am     gear
## 23.500  4.000 400.900 283.000  2.170  3.911  8.400  1.000  1.000  2.000
##      carb
##      7.000
```

Con la sintaxis abreviada de purrr:

```
map_dbl(mtcars, ~ max(.) - min(.))
```

```
##      mpg      cyl    disp      hp    drat      wt     qsec      vs      am     gear
## 23.500  4.000 400.900 283.000  2.170  3.911  8.400  1.000  1.000  2.000
##      carb
##      7.000
```

En esta segunda forma:

- ~ indica que estamos creando una función anónima;
- . representa el elemento actual;
- max(.) - min(.) se aplica a cada columna.

1.8 Pasar argumentos adicionales

Algunas funciones necesitan argumentos adicionales.

Por ejemplo, supongamos que tenemos valores faltantes.

```
df <- tibble(
  a = c(1, 2, NA, 4),
  b = c(10, NA, 30, 40),
  c = c(5, 6, 7, NA)
)

df
```

```
## # A tibble: 4 x 3
##       a      b      c
##   <dbl> <dbl> <dbl>
## 1     1    10     5
## 2     2     NA     6
## 3    NA    30     7
## 4     4    40    NA
```

Si usamos mean() directamente:

```
map_dbl(df, mean)
```



```
## a b c
## NA NA NA
```

El resultado contiene NA, porque las columnas tienen valores faltantes.

Podemos pasar el argumento `na.rm = TRUE`:

```
map_dbl(df, mean, na.rm = TRUE)
```

```
## a b c
## 2.333333 26.666667 6.000000
```

Esto significa:

aplica `mean()` a cada columna, pero en cada aplicación usa también `na.rm = TRUE`.

Es equivalente a escribir:

```
mean(df$a, na.rm = TRUE)
mean(df$b, na.rm = TRUE)
mean(df$c, na.rm = TRUE)
```

1.9 Extraer elementos por nombre

Una de las ventajas de `map()` es que también permite trabajar con listas complejas.

Por ejemplo, ajustemos un modelo lineal para cada grupo de cilindros en `mtcars`.

```
modelos <- mtcars %>%
  split(.$cyl) %>%
  map(~ lm(mpg ~ wt, data = .))
```

```
modelos
```

```
## $'4'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Coefficients:
## (Intercept) wt
## 39.571 -5.647
##
## $'6'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Coefficients:
## (Intercept) wt
## 28.41 -2.78
```

```
##
##
## $'8'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Coefficients:
## (Intercept)          wt
##      23.868      -2.192
```

Aquí ocurre lo siguiente:

1. `split(.$cyl)` divide `mtcars` según el número de cilindros.
2. `map()` aplica un modelo lineal a cada subconjunto.
3. En cada subconjunto se ajusta `lm(mpg ~ wt, data = .)`.

Ahora podemos obtener el resumen de cada modelo.

```
resumenes <- map(modelos, summary)
```

```
resumenes
```

```
## $'4'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.1513 -1.9795 -0.6272  1.9299  5.2523
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   39.571      4.347    9.104 7.77e-06 ***
## wt           -5.647      1.850   -3.052  0.0137 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.332 on 9 degrees of freedom
## Multiple R-squared:  0.5086, Adjusted R-squared:  0.454
## F-statistic: 9.316 on 1 and 9 DF,  p-value: 0.01374
##
##
## $'6'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Residuals:
##      Mazda RX4  Mazda RX4 Wag  Hornet 4 Drive      Valiant      Merc 280
##      -0.1250      0.5840      1.9292     -0.6897      0.3547
##      Merc 280C   Ferrari Dino
```

```
##           -1.0453           -1.0080
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  28.409      4.184   6.789 0.00105 **
## wt          -2.780      1.335  -2.083 0.09176 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.165 on 5 degrees of freedom
## Multiple R-squared:  0.4645, Adjusted R-squared:  0.3574
## F-statistic: 4.337 on 1 and 5 DF, p-value: 0.09176
##
##
## $'8'
##
## Call:
## lm(formula = mpg ~ wt, data = .)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.1491 -1.4664 -0.8458  1.5711  3.7619
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  23.8680      3.0055   7.942 4.05e-06 ***
## wt          -2.1924      0.7392  -2.966  0.0118 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.024 on 12 degrees of freedom
## Multiple R-squared:  0.423, Adjusted R-squared:  0.3749
## F-statistic: 8.796 on 1 and 12 DF, p-value: 0.01179
```

Si queremos extraer el valor de `r.squared` de cada resumen, podemos escribir:

```
map_dbl(resumenes, "r.squared")
```

```
##           4           6           8
## 0.5086326 0.4645102 0.4229655
```

La expresión:

```
map_dbl(resumenes, "r.squared")
```

significa:

de cada elemento de la lista `resumenes`, extrae el componente llamado `r.squared`.

También podríamos escribirlo así:

```
map_dbl(resumenes, ~ .$r.squared)
```

```
##           4           6           8
## 0.5086326 0.4645102 0.4229655
```

Ambas formas dan el mismo resultado.

1.10 Comparación entre for, lapply() y map()

Las tres opciones siguientes hacen una tarea similar.

Con for:

```
salida <- vector("double", length(mtcars))
names(salida) <- names(mtcars)

for (i in seq_along(mtcars)) {
  salida[[i]] <- mean(mtcars[[i]])
}

salida
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Con lapply():

```
lapply(mtcars, mean)
```

```
## $mpg
## [1] 20.09062
##
## $cyl
## [1] 6.1875
##
## $disp
## [1] 230.7219
##
## $hp
## [1] 146.6875
##
## $drat
## [1] 3.596563
##
## $wt
## [1] 3.21725
##
## $qsec
## [1] 17.84875
```

```
##
## $vs
## [1] 0.4375
##
## $am
## [1] 0.40625
##
## $gear
## [1] 3.6875
##
## $carb
## [1] 2.8125
```

Con `map_dbl()`:

```
map_dbl(mtcars, mean)
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

La diferencia principal es la salida:

- `for` permite mucho control, pero requiere más código.
- `lapply()` siempre devuelve una lista.
- `map_dbl()` devuelve directamente un vector numérico.

En ciencia de datos, `map()` y sus variantes son muy útiles porque permiten escribir iteraciones de forma compacta y clara.

2. Manejo de errores al iterar

Cuando aplicamos una función muchas veces, puede pasar que una de las iteraciones falle.

Por ejemplo, la función `log()` calcula logaritmos, pero necesita valores numéricos.

```
log(10)
```

```
## [1] 2.302585
```

Pero si intentamos calcular:

```
log("a")
```

```
## Error in 'log()':
## ! Argumento no numérico para una función matemática
```

obtenemos un error, porque "a" no es un valor numérico.

Ahora imaginemos que tenemos una lista con varios elementos:

```
x <- list(1, 10, "a", 100)
```

Si intentamos aplicar `log()` con `map()`:

```
map(x, log)
```

```
## Error in 'map()':  
## i In index: 3.  
## Caused by error in '.f()':  
## ! Argumento no numérico para una función matemática
```

El problema es que una sola entrada mala, "a", detiene todo el proceso.

Para evitar esto, **purrr** ofrece herramientas para manejar errores.

2.1 La función `safely()`

La función `safely()` transforma una función normal en una función segura.

Por ejemplo:

```
log_seguro <- safely(log)
```

Ahora `log_seguro()` no se detiene cuando ocurre un error.

```
log_seguro(10)
```

```
## $result  
## [1] 2.302585  
##  
## $error  
## NULL
```

```
log_seguro("a")
```

```
## $result  
## NULL  
##  
## $error  
## <simpleError in .f(...): Argumento no numérico para una función matemática>
```

La salida de `safely()` siempre es una lista con dos elementos:

- **result**: contiene el resultado si la función funcionó;
- **error**: contiene el error si la función falló.

Si no hubo error, **error** será `NULL`.

Si hubo error, **result** será `NULL`.

2.2 Usar safely() con map()

Ahora apliquemos `log_seguro()` a toda la lista `x`.

```
x <- list(1, 10, "a", 100)

resultado <- map(x, log_seguro)

resultado
```

```
## [[1]]
## [[1]]$result
## [1] 0
##
## [[1]]$error
## NULL
##
##
## [[2]]
## [[2]]$result
## [1] 2.302585
##
## [[2]]$error
## NULL
##
##
## [[3]]
## [[3]]$result
## NULL
##
## [[3]]$error
## <simpleError in .f(...): Argumento no numérico para una función matemática>
##
##
## [[4]]
## [[4]]$result
## [1] 4.60517
##
## [[4]]$error
## NULL
```

Cada elemento de `resultado` contiene una lista con `result` y `error`.

Podemos observar su estructura:

```
str(resultado)

## List of 4
## $ :List of 2
## ..$ result: num 0
## ..$ error : NULL
## $ :List of 2
## ..$ result: num 2.3
```

```
## ..$ error : NULL
## $ :List of 2
## ..$ result: NULL
## ..$ error :List of 2
## .. ..$ message: chr "Argumento no numérico para una función matemática"
## .. ..$ call : language .f(...)
## .. ..- attr(*, "class")= chr [1:3] "simpleError" "error" "condition"
## $ :List of 2
## ..$ result: num 4.61
## ..$ error : NULL
```

Este tipo de salida es útil porque no perdemos los resultados que sí funcionaron.

2.3 Separar resultados y errores

La función `transpose()` permite reorganizar la salida.

```
resultado_transpuesto <- transpose(resultado)

resultado_transpuesto
```

```
## $result
## $result[[1]]
## [1] 0
##
## $result[[2]]
## [1] 2.302585
##
## $result[[3]]
## NULL
##
## $result[[4]]
## [1] 4.60517
##
##
## $error
## $error[[1]]
## NULL
##
## $error[[2]]
## NULL
##
## $error[[3]]
## <simpleError in .f(...): Argumento no numérico para una función matemática>
##
## $error[[4]]
## NULL
```

Ahora tenemos una lista con dos partes principales:

- una lista de resultados;
- una lista de errores.

Podemos extraer los resultados:

```
resultado_transpuesto$result
```

```
## [[1]]  
## [1] 0  
##  
## [[2]]  
## [1] 2.302585  
##  
## [[3]]  
## NULL  
##  
## [[4]]  
## [1] 4.60517
```

Y también los errores:

```
resultado_transpuesto$error
```

```
## [[1]]  
## NULL  
##  
## [[2]]  
## NULL  
##  
## [[3]]  
## <simpleError in .f(...): Argumento no numérico para una función matemática>  
##  
## [[4]]  
## NULL
```

2.4 Identificar qué elementos funcionaron

Podemos detectar cuáles iteraciones no produjeron error.

```
funciono <- map_lgl(resultado_transpuesto$error, is.null)
```

```
funciono
```

```
## [1] TRUE TRUE FALSE TRUE
```

Esto devuelve TRUE cuando no hubo error y FALSE cuando sí hubo error.

Podemos ver los elementos originales que fallaron:

```
x[!funciono]
```

```
## [[1]]  
## [1] "a"
```

También podemos ver los resultados que sí funcionaron:

```
resultado_transpuesto$result[funciono]
```

```
## [[1]]  
## [1] 0  
##  
## [[2]]  
## [1] 2.302585  
##  
## [[3]]  
## [1] 4.60517
```

Si queremos convertirlos en un vector numérico:

```
resultado_transpuesto$result[funciono] %>%  
  flatten_dbl()
```

```
## [1] 0.000000 2.302585 4.605170
```

2.5 La función possibly()

La función `possibly()` es una alternativa más simple a `safely()`.

En lugar de devolver el error, permite indicar un valor por defecto cuando ocurre un error.

Por ejemplo:

```
log_posible <- possibly(log, otherwise = NA_real_)
```

Esto significa:

intenta aplicar `log()`, pero si ocurre un error, devuelve `NA_real_`.

Ahora:

```
log_posible(10)
```

```
## [1] 2.302585
```

```
log_posible("a")
```

```
## [1] NA
```

Si lo usamos con `map_dbl()`:

```
x <- list(1, 10, "a", 100)  
map_dbl(x, log_posible)
```

```
## [1] 0.000000 2.302585 NA 4.605170
```

Aquí obtenemos un vector numérico. En la posición correspondiente a "a" aparece NA.

Esta estrategia es muy útil cuando queremos continuar el análisis aunque algunos elementos fallen.

2.6 Diferencia entre `safely()` y `possibly()`

La diferencia principal es esta:

Función	¿Qué hace cuando hay error?
<code>safely()</code>	Guarda el resultado y también el error
<code>possibly()</code>	Devuelve un valor por defecto

Usamos `safely()` cuando queremos inspeccionar qué salió mal.

Usamos `possibly()` cuando solo queremos reemplazar los errores por un valor definido, como `NA`.

2.7 La función `quietly()`

A veces una función no produce errores, pero sí genera advertencias, mensajes o impresiones en pantalla.

Por ejemplo:

```
log(-1)
```

```
## Warning in log(-1): Se han producido NaNs
```

```
## [1] NaN
```

Esto no produce un error como tal, pero sí produce una advertencia porque el logaritmo de un número negativo no es real en los números reales.

La función `quietly()` captura:

- resultado;
- salida impresa;
- advertencias;
- mensajes.

```
log_silencioso <- quietly(log)
```

```
log_silencioso(10)
```

```
## $result
## [1] 2.302585
##
## $output
## [1] ""
##
## $warnings
## character(0)
##
## $messages
## character(0)
```

```
log_silencioso(-1)
```

```
## $result
## [1] NaN
##
## $output
## [1] ""
##
## $warnings
## [1] "Se han producido NaNs"
##
## $messages
## character(0)
```

La salida contiene varios elementos:

- **result**: resultado de la función;
- **output**: texto impreso;
- **warnings**: advertencias;
- **messages**: mensajes.

2.8 Usar quietly() con map()

Consideremos esta lista:

```
valores <- list(1, -1, 10, -5)
```

Aplicamos log() de forma silenciosa:

```
salida <- map(valores, quietly(log))
```

```
salida
```

```
## [[1]]
## [[1]]$result
## [1] 0
##
## [[1]]$output
## [1] ""
##
## [[1]]$warnings
## character(0)
##
## [[1]]$messages
## character(0)
##
##
## [[2]]
## [[2]]$result
## [1] NaN
##
```

```
## [[2]]$output
## [1] ""
##
## [[2]]$warnings
## [1] "Se han producido NaNs"
##
## [[2]]$messages
## character(0)
##
##
## [[3]]
## [[3]]$result
## [1] 2.302585
##
## [[3]]$output
## [1] ""
##
## [[3]]$warnings
## character(0)
##
## [[3]]$messages
## character(0)
##
##
## [[4]]
## [[4]]$result
## [1] NaN
##
## [[4]]$output
## [1] ""
##
## [[4]]$warnings
## [1] "Se han producido NaNs"
##
## [[4]]$messages
## character(0)
```

Podemos extraer las advertencias:

```
map(salida, "warnings")
```

```
## [[1]]
## character(0)
##
## [[2]]
## [1] "Se han producido NaNs"
##
## [[3]]
## character(0)
##
## [[4]]
## [1] "Se han producido NaNs"
```

Y podemos extraer los resultados:

```
map(salida, "result")
```

```
## [[1]]
## [1] 0
##
## [[2]]
## [1] NaN
##
## [[3]]
## [1] 2.302585
##
## [[4]]
## [1] NaN
```

Esto es útil cuando queremos revisar qué iteraciones generaron advertencias sin interrumpir el flujo de trabajo.

3. Ejercicios

Ejercicio 1

Usa `map_dbl()` para calcular la media de cada columna de `mtcars`.

```
map_dbl(mtcars, mean)
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 2

Usa `map_dbl()` para calcular la desviación estándar de cada columna de `mtcars`.

```
map_dbl(mtcars, sd)
```

```
##      mpg      cyl      disp      hp      drat      wt
##  6.0269481  1.7859216 123.9386938 68.5628685  0.5346787  0.9784574
##      qsec      vs      am      gear      carb
##  1.7869432  0.5040161  0.4989909  0.7378041  1.6152000
```

Ejercicio 3

Usa `map_chr()` para obtener la clase de cada columna de `iris`.

```
map_chr(iris, class)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##      "numeric"      "numeric"      "numeric"      "numeric"      "factor"
```

Ejercicio 4

Usa `map_lgl()` para determinar qué columnas de `iris` son numéricas.

```
map_lgl(iris, is.numeric)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##           TRUE           TRUE           TRUE           TRUE      FALSE
```

Ejercicio 5

Usa `map_int()` para calcular el número de valores únicos en cada columna de `iris`.

```
map_int(iris, ~ length(unique(.)))
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##           35           23           43           22           3
```

Ejercicio 6

Usa `map_dbl()` para calcular el rango de cada columna de `mtcars`, definido como:

$$\max(x) - \min(x).$$

```
map_dbl(mtcars, ~ max(.) - min(.))
```

```
##      mpg      cyl    disp      hp      drat      wt      qsec      vs      am      gear
## 23.500  4.000 400.900 283.000  2.170  3.911  8.400  1.000  1.000  2.000
##      carb
##  7.000
```

Ejercicio 7

Crea una lista con los valores 1, 10, "a" y 100. Usa `safely()` para aplicar `log()` sin que el proceso se detenga.

```
lista <- list(1,10,"a",100)
log_seguro <- safely(log)
resultado <- map(lista,log_seguro)
resultado
```

```
## [[1]]
## [[1]]$result
## [1] 0
##
## [[1]]$error
## NULL
##
##
```

```
## [[2]]
## [[2]]$result
## [1] 2.302585
##
## [[2]]$error
## NULL
##
##
## [[3]]
## [[3]]$result
## NULL
##
## [[3]]$error
## <simpleError in .f(...): Argumento no numérico para una función matemática>
##
##
## [[4]]
## [[4]]$result
## [1] 4.60517
##
## [[4]]$error
## NULL
```

Ejercicio 8

Usa el resultado del ejercicio anterior para identificar cuáles elementos produjeron error.

```
resultado_transpuesto <- transpose(resultado)
error_ <- map_lgl(resultado_transpuesto$result,is.null)
error_
```

```
## [1] FALSE FALSE TRUE FALSE
```

```
lista[error_]
```

```
## [[1]]
## [1] "a"
```

Ejercicio 9

Usa possibly() para aplicar log() a la lista x <- list(1, 10, "a", 100). Cuando ocurra un error, debe devolver NA.

```
log_possibly <- possibly(log,otherwise = NA_real_)
map_dbl(lista,log_possibly)
```

```
## [1] 0.000000 2.302585 NA 4.605170
```


Ejercicio 10

Usa `quietly()` para aplicar `log()` a los valores 1, -1, 10 y -5. Luego extrae las advertencias generadas.

```
lista2 <- list(1,-1,10,-5)
salida <- map(lista2, quietly(log))
map(salida, "warnings")
```

```
## [[1]]
## character(0)
##
## [[2]]
## [1] "Se han producido NaNs"
##
## [[3]]
## character(0)
##
## [[4]]
## [1] "Se han producido NaNs"
```